Reporte

Nombre: 1. Roberto Santana Trujillo 2. Isaías Leonardo Salazar Torres	**Matrícula**: 2227084 2227016
Nombre del curso: Componentes de lógica de negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 4	**Actividad**: U4 – Laboratorio guiado, programa para TechStore.
Fecha: 28/10/2026	
Bibliografia (Formato APA) Presentación mostrada en clase.	

Objetivo:

Implementar el núcleo de negocio de TechStore a partir de los archivos .csv y las reglas dadas por la unidad 4. Desarrollar entidades y value objects con invariantes, especificaciones combinables, una máquina de estados con domain events e implementar una API REST.

En este informe recopila lo realizado para cumplir las reglas de TechStore y las pruebas realizadas para comprobar que el sistema es correcto. El código ejecutable, los archivos originales y las evidencias completas se conservan en el repositorio de GitHub:

https://github.com/ileosalazar/techstore-u4

Procedimiento

1: Organización del repositorio, estructura y dominio

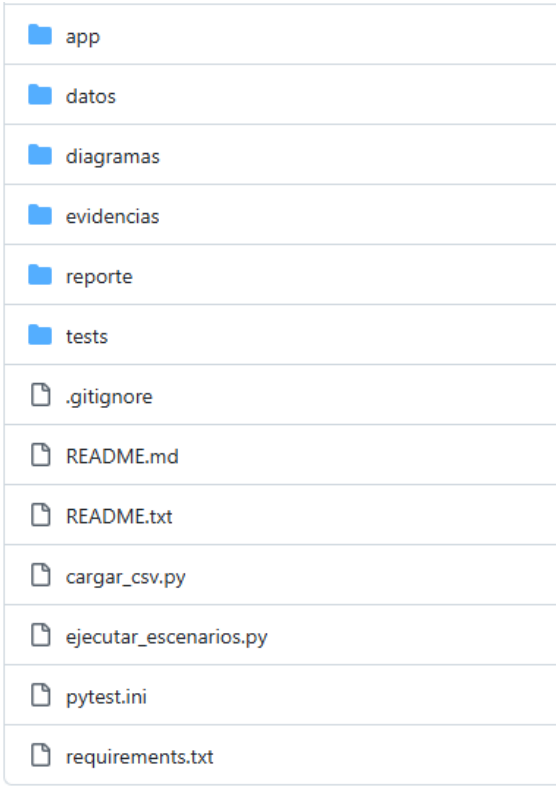
El repositorio creado y alojado en Github contiene las carpetas app, tests, datos, diagramas, evidencias y reporte. El archivo .gitignore excluye el entorno virtual y los archivos de caché. El flujo automatizado de GitHub ejecuta carga, escenarios y pytest con Python 3.12, para poder implementar la API y la interfaz del sistema.

Además, se agregó un archivo README para acompañar el código con las instrucciones que deben seguirse para ejecutar las pruebas y el sistema, desde la instalación hasta los endpoints del sistema.

Por último, fuera de las carpetas se agregan los archivos de cargar_csv.py y otros necesarios para ejecutar las pruebas.

La estructura del repositorio en GitHub es la siguiente:

app/domain.py Entidades, VO, estados y eventos
app/specifications.py Reglas AND, OR y NOT; descuentos
app/api.py Adaptador HTTP y aplicación FastAPI
datos/ Los cinco CSV originales del inge
tests/ Dominio, reglas, API y escenarios
diagramas/ Draw.io editable y su PNG exportado
evidencias/ Bitácora, logs, OpenAPI y capturas
cargar_csv.py Cargador original del paquete
ejecutar_escenarios.py Ejecución reproducible de E01 a E15
requirements.txt Versiones verificadas del entorno



2. README con instalación, ejecución, reglas y endpoints.

El siguiente README ya existe en el repositorio GitHub con el nombre de README.md. Aquí se encuentran las instrucciones para instalar el entorno virtual, ejecutarlo con la API, las reglas del negocio y los endpoints del mismo. Se transcribe y luego se adjuntan capturas de cómo aparece en el repositorio.

TechStore U4

Implementa pedidos, reglas de descuento, una máquina de estados, eventos de dominio y una API REST sobre los CSV originales. La validación produjo 90 pruebas aprobadas y 15 de 15 escenarios verificados.

Integrantes: Roberto Santana Trujillo (2227084) e Isaías Leonardo Salazar Torres (2227016).

Repositorio: https://github.com/ileosalazar/techstore-u4

Instalación:

Requiere Python 3.11 o superior; se verificó con Python 3.12.14. Clona el repositorio y entra a su carpeta.

Bash:

```
git clone https://github.com/Santana-R/techstore-u4.git
cd techstore-u4
python -m venv .venv
```

Activa el entorno según tu sistema. En macOS o Linux:

Bash:

```
source .venv/bin/activate
```

En Windows PowerShell:

PowerShell:

```
.venv\Scripts\Activate.ps1
```

En Windows CMD:

Bat:

.venv\Scripts\activate.bat

Instala las versiones verificadas:

Bash:

python -m pip install -r requirements.txt

Si PowerShell bloquea la activación, usa directamente `.venv\Scripts\python.exe` en lugar de `python`. En macOS o Linux, usa `.venv/bin/python`. La instalación ocurre dentro del entorno virtual.

Ejecución y comprobación:

Ejecuta desde la raíz del proyecto:

Bash:

python cargar_csv.py

python ejecutar_escenarios.py

python -m pytest -q

python -m uvicorn app.api:app --reload

El cargador muestra los 16 pedidos en DRAFT. O001 debe tener 4 unidades y 2100.00 MXN. El ejecutor verifica E01 a E15 con datos nuevos por caso y actualiza `evidencias/escenarios.json` y `evidencias/bitacora.csv`. pytest debe informar 90 pruebas aprobadas.

Swagger se abre en http://127.0.0.1:8000/docs y el contrato OpenAPI está en http://127.0.0.1:8000/openapi.json. Las capturas del reporte usaron el puerto 8764; es la misma aplicación. Detén el servidor con Ctrl+C. Al reiniciarlo, todos los pedidos vuelven a DRAFT.

Estructura:

app/domain.py Entidades, VO, estados y eventos

app/specifications.py Reglas AND, OR y NOT; descuentos

app/api.py Adaptador HTTP y aplicación FastAPI

datos/ Los cinco CSV originales del inge

tests/ Dominio, reglas, API y escenarios

diagramas/ Draw.io editable y su PNG exportado

evidencias/ Bitácora, logs, OpenAPI y capturas

cargar_csv.py Cargador original del paquete

ejecutar_escenarios.py Ejecución reproducible de E01 a E15

requirements.txt Versiones verificadas del entorno

`.venv`, `__pycache__` y `.pytest_cache` se excluyen del repositorio. Los CSV y el cargador se conservaron sin cambios.

Reglas de negocio:

- Money es inmutable, utiliza Decimal y rechaza importes negativos, no finitos y valores float. add exige la misma moneda. Los pedidos usan MXN.

- OrderItem es inmutable y exige SKU y cantidad entera positiva. El máximo es 20 unidades sumadas, aunque estén en uno o varios renglones.

- Descuento del 10 por ciento: cliente VIP, total bruto estrictamente mayor que 500 MXN, al menos 4 unidades y sin promoción.

- Descuento del 5 por ciento: total bruto estrictamente mayor que 5000 MXN y sin promoción. Si ambas reglas se cumplen, se aplica solo 10 por ciento.

- El descuento se calcula sobre el total bruto. El importe descontado se redondea a centavos con ROUND_HALF_UP; consultar el descuento no modifica el pedido ni genera eventos.

- Confirmar exige DRAFT, artículos y cliente activo. Enviar exige CONFIRMED y guía. Cancelar exige CONFIRMED y motivo. Entregar y devolver exigen SHIPPED.

- Toda transición válida registra un evento con pedido, cliente, total y fecha UTC. Un rechazo conserva el estado, los artículos y los eventos existentes.

- La existencia de productos es contexto del catálogo. No hay control de inventario, base de datos ni envío real de mensajes. Los objetos se guardan en memoria y los eventos se consultan en la respuesta de cada pedido.

Endpoints:

Todos los métodos devuelven 200 cuando la operación es válida. Los importes se serializan como cadenas decimales para conservar su representación.

Método	Ruta	Entrada o resultado
GET	/orders/{order_id}	Consulta el pedido y sus eventos
POST	/orders/{order_id}/confirm	Confirma sin cuerpo JSON
POST	/orders/{order_id}/ship	tracking_number como cadena
POST	/orders/{order_id}/cancel	reason como cadena
POST	/orders/{order_id}/deliver	Entrega desde SHIPPED
POST	/orders/{order_id}/return	Devuelve desde SHIPPED
POST	/orders/{order_id}/items	sku y quantity; precio del CSV

404 identifica pedido o SKU inexistente; 409 indica una transición prohibida; 422 indica datos o una regla de negocio inválidos. Los cuerpos rechazan campos adicionales y cantidades que no sean enteros positivos.

Ejemplo en Swagger

1. En POST /orders/{order_id}/confirm, pulsa Try it out y escribe O013. Execute devuelve 200, CONFIRMED y OrderConfirmed.

2. Ejecuta otra vez el mismo pedido: devuelve 409 y conserva un solo evento.

3. Usa O007 para comprobar 422 por pedido vacío y NOEXISTE para comprobar 404.

4. Reinicia el servidor para repetir los casos desde DRAFT. Las pruebas automáticas crean una aplicación independiente por caso.

Evidencias y diagrama

`evidencias/carga.txt`, `pytest.txt` y `escenarios.txt` conservan la salida real de ejecución. Sus capturas se muestran en visores locales; los HTML identifican expresamente el comando y el archivo de origen. Las imágenes swagger_200, swagger_404, swagger_409 y swagger_422 son capturas reales de /docs.

El archivo `diagramas/techstore_estados.drawio` es editable en https://app.diagrams.net. Su PNG se exportó desde draw.io e incluye una copia editable del diagrama. Los recortes del reporte solo encuadran los resultados; las capturas completas se conservan en el repositorio.

README.md en el repositorio:

TechStore U4

Implementa pedidos, reglas de descuento, una máquina de estados, eventos de dominio y una API REST sobre los CSV originales. La validación produjo 90 pruebas aprobadas y 15 de 15 escenarios verificados.

Integrantes: Roberto Santana Trujillo (2227084) e Isaías Leonardo Salazar Torres (2227016).

Repositorio: https://github.com/ileosalazar/techstore-u4

Instalación

Requiere Python 3.11 o superior; se verificó con Python 3.12.14. Clona el repositorio y entra a su carpeta.

```
git clone https://github.com/ileosalazar/techstore-u4
cd techstore-u4
python -m venv .venv
```

Activa el entorno según tu sistema. En macOS o Linux:

```
source .venv/bin/activate
```

En Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

En Windows CMD:

```
.venv\Scripts\activate.bat
```

Instala las versiones verificadas:

```
python -m pip install -r requirements.txt
```

Si PowerShell bloquea la activación, usa directamente `.venv\Scripts\python.exe` en lugar de `python`. En macOS o Linux, usa `.venv/bin/python`. La instalación ocurre dentro del entorno virtual.

Ejecución y comprobación

Ejecuta desde la raíz del proyecto:

```
python cargar_csv.py
python ejecutar_escenarios.py
python -m pytest -q
python -m uvicorn app.api:app --reload
```

El cargador muestra los 16 pedidos en DRAFT. O001 debe tener 4 unidades y 2100.00 MXN. El ejecutor verifica E01 a E15 con datos nuevos por caso y actualiza `evidencias/escenarios.json` y `evidencias/bitacora.csv`. pytest debe informar 90 pruebas aprobadas.

Swagger se abre en http://127.0.0.1:8000/docs y el contrato OpenAPI está en http://127.0.0.1:8000/openapi.json. Las capturas del reporte usaron el puerto 8764; es la misma aplicación. Detén el servidor con Ctrl+C. Al reiniciarlo, todos los pedidos vuelven a DRAFT.

Estructura

```
app/domain.py           Entidades, VO, estados y eventos
app/specifications.py   Reglas AND, OR y NOT; descuentos
app/api.py              Adaptador HTTP y aplicación FastAPI
datos/                  Los cinco CSV originales del inge
tests/                  Dominio, reglas, API y escenarios
diagramas/              Draw.io editable y su PNG exportado
evidencias/             Bitácora, logs, OpenAPI y capturas
cargar_csv.py           Cargador original del paquete
ejecutar_escenarios.py  Ejecución reproducible de E01 a E15
requirements.txt        Versiones verificadas del entorno
```

`.venv`, `__pycache__` y `.pytest_cache` se excluyen del repositorio. Los CSV y el cargador se conservaron sin cambios.

Reglas de negocio

- Money es inmutable, utiliza Decimal y rechaza importes negativos, no finitos y valores float. add exige la misma moneda. Los pedidos usan MXN.
- OrderItem es inmutable y exige SKU y cantidad entera positiva. El máximo es 20 unidades sumadas, aunque estén en uno o varios renglones.
- Descuento del 10 por ciento: cliente VIP, total bruto estrictamente mayor que 500 MXN, al menos 4 unidades y sin promoción.
- Descuento del 5 por ciento: total bruto estrictamente mayor que 5000 MXN y sin promoción. Si ambas reglas se cumplen, se aplica solo 10 por ciento.
- El descuento se calcula sobre el total bruto. El importe descontado se redondea a centavos con ROUND_HALF_UP; consultar el descuento no modifica el pedido ni genera eventos.
- Confirmar exige DRAFT, artículos y cliente activo. Enviar exige CONFIRMED y guía. Cancelar exige CONFIRMED y motivo. Entregar y devolver exigen SHIPPED.
- Toda transición válida registra un evento con pedido, cliente, total y fecha UTC. Un rechazo conserva el estado, los artículos y los eventos existentes.
- La existencia de productos es contexto del catálogo. No hay control de inventario, base de datos ni envío real de mensajes. Los objetos se guardan en memoria y los eventos se consultan en la respuesta de cada pedido.

Endpoints

Todos los métodos devuelven 200 cuando la operación es válida. Los importes se serializan como cadenas decimales para conservar su representación.

Método	Ruta	Entrada o resultado
GET	/orders/{order_id}	Consulta el pedido y sus eventos
POST	/orders/{order_id}/confirm	Confirma sin cuerpo JSON
POST	/orders/{order_id}/ship	tracking_number como cadena
POST	/orders/{order_id}/cancel	reason como cadena
POST	/orders/{order_id}/deliver	Entrega desde SHIPPED
POST	/orders/{order_id}/return	Devuelve desde SHIPPED
POST	/orders/{order_id}/items	sku y quantity; precio del CSV

404 identifica pedido o SKU inexistente; 409 indica una transición prohibida; 422 indica datos o una regla de negocio inválidos. Los cuerpos rechazan campos adicionales y cantidades que no sean enteros positivos.

Ejemplo en Swagger

1. En POST /orders/{order_id}/confirm, pulsa Try it out y escribe O013. Execute devuelve 200, CONFIRMED y OrderConfirmed.
2. Ejecuta otra vez el mismo pedido: devuelve 409 y conserva un solo evento.
3. Usa O007 para comprobar 422 por pedido vacío y NOEXISTE para comprobar 404.
4. Reinicia el servidor para repetir los casos desde DRAFT. Las pruebas automáticas crean una aplicación independiente por caso.

Evidencias y diagrama

`evidencias/carga.txt`, `pytest.txt` y `escenarios.txt` conservan la salida real de ejecución. Sus capturas se muestran en visores locales; los HTML identifican expresamente el comando y el archivo de origen. Las imágenes swagger_200, swagger_404, swagger_409 y swagger_422 son capturas reales de /docs.

El archivo `diagramas/techstore_estados.drawio` es editable en https://app.diagrams.net. Su PNG se exportó desde draw.io e incluye una copia editable del diagrama. Los recortes del reporte solo encuadran los resultados; las capturas completas se conservan en el repositorio.

3. Carga exitosa de los 16 pedidos

Se cargaron los 16 pedidos desde los CSV originales. Todos comienzan DRAFT; O007 se conserva vacío para comprobar su rechazo al confirmar. Estos pedidos se cargan al ingresar al entorno virtual y ejecutar “python cargar_csv.py”.

```
(.venv) PS C:\Users\salaz\Desktop\techstore-u4> python cargar_csv.py
0001 DRAFT 4 2100.00
0002 DRAFT 2 1050.00
0003 DRAFT 4 2100.00
0004 DRAFT 1 850.00
0005 DRAFT 4 2100.00
0006 DRAFT 1 18000.00
0007 DRAFT 0 0
0008 DRAFT 20 2400.00
0009 DRAFT 19 2280.00
0010 DRAFT 2 500.00
0011 DRAFT 3 1500.00
0012 DRAFT 1 850.00
0013 DRAFT 1 600.00
0014 DRAFT 1 600.00
0015 DRAFT 1 450.00
0016 DRAFT 1 2100.00
(.venv) PS C:\Users\salaz\Desktop\techstore-u4>
```

4. Bitácora de escenarios E01 a E15

Cada escenario se ejecutó con load_fixture() nuevo. La prueba parametrizada tests/test_scenarios.py verifica resultados concretos y comprueba que una operación rechazada conserve estado, artículos y eventos.

Caso	Estado inicial	Operación	Resultado real	Explicación
E01 Pedido O001	DRAFT; cliente C001, VIP, activo; 4 unidades; total 2100.00 MXN; sin promoción	evaluar_descuento.	DRAFT. Cumple VIP, total >500, al menos 4 unidades y ausencia de promoción. El descuento es 210.00 y el neto 1890.00 MXN. test_escenario[E01] aprobada.	El pedido cumple con lo necesario para aplicarse un descuento del 10% como lo muestran las reglas del negocio.
E02 Pedido O002	DRAFT; cliente C002, no VIP, activo; 2 unidades; total 1050.00 MXN; sin promoción.	evaluar_descuento.	DRAFT. El cliente no es VIP y 1050.00 no supera 5000. No se concede descuento. test_escenario[E02] aprobada.	No cumple con lo necesario para aplicarse descuento según las reglas del negocio.

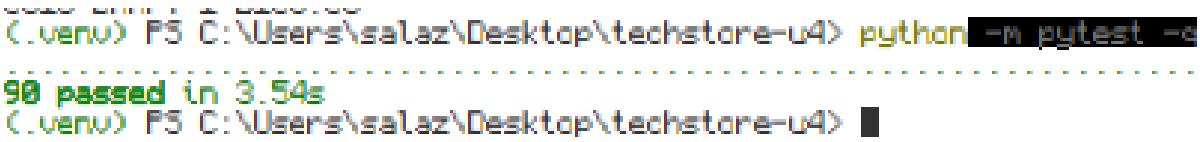
E03 Pedido O003	DRAFT; cliente C003, VIP, inactivo; 4 unidades; total 2100.00 MXN; sin promoción.	confirmar.	InvalidOrderData; DRAFT. El cliente C003 está inactivo. Se rechaza confirmar y el pedido sigue en DRAFT.	No se puede confirmar el pedido debido a que el cliente no está activo, por lo que se rechaza pero se mantiene el pedido.
E04 Pedido O005	DRAFT; cliente C005, VIP, activo; 4 unidades; total 2100.00 MXN; con promoción.	evaluar_descuento.	DRAFT. La promoción activa bloquea ambos descuentos, aunque el cliente sea VIP y reúna monto y unidades. test_escenario[E04] aprobada.	Al haber una promoción activa, no se aplica ningún descuento al pedido.
E05 Pedido O006	DRAFT; cliente C006, no VIP, activo; 1 unidades; total 18000.00 MXN; sin promoción.	evaluar_descuento.	DRAFT. 18000.00 supera 5000 y no hay promoción. Descuento de 900.00 y neto de 17100.00 MXN. test_escenario[E05] aprobada.	Se aplica 5% de descuento al cumplir con ambas reglas, pero al ser solo una unidad y no VIP, no se aplica el 10%.
E06 Pedido O007	DRAFT; cliente C007, VIP, activo; 0 unidades; total 0.00 MXN; sin promoción.	confirmar.	InvalidOrderData; DRAFT. No tiene artículos. La confirmación se rechaza sin generar evento. test_escenario[E06] aprobada.	No se confirma el pedido debido a que no tiene ningún artículo. Se rechaza y error 422.
E07 Pedido O008	DRAFT; cliente C008, no VIP, activo; 20 unidades; total 2400.00 MXN; sin promoción.	confirmar.	CONFIRMED. Las 20 unidades exactas están permitidas. Confirmar cambia el estado a CONFIRMED. test_escenario[E07] aprobada.	Se confirma el pedido al estar dentro de los límites permitidos.

E08 Pedido O009	DRAFT; cliente C009, VIP, activo; 19 unidades; total 2280.00 MXN; sin promoción.	agregar SKU003 x2.	OrderItemLimitExceeded; DRAFT. 19 + 2 serían 21 unidades. Se rechaza antes de añadir: se conservan 19 unidades y 2280.00 MXN. test_escenario[E08] aprobada.	Se rechaza el pedido antes de añadir unidades que sobrepasen el límite de 20 establecido por las reglas.
E09 Pedido O010	DRAFT; cliente C010, no VIP, activo; 2 unidades; total 500.00 MXN; sin promoción.	evaluar_descuento.	DRAFT. El total es exactamente 500.00; no cumple el umbral estricto >500 ni la alternativa >5000. test_escenario[E09] aprobada.	No se aplica el descuento al no cumplir la regla de ser más de 500 MXN.
E10 Pedido O011	DRAFT; cliente C011, VIP, activo; 3 unidades; total 1500.00 MXN; sin promoción.	evaluar_descuento.	DRAFT. Es VIP y supera 500, pero tiene 3 unidades. No cumple el mínimo de 4 y tampoco supera 5000. test_escenario[E10] aprobada.	No aplica ningún descuento al no tener el mínimo de unidades.
E11 Pedido O012	DRAFT; cliente C012, no VIP, activo; 1 unidades; total 850.00 MXN; sin promoción.	confirmar y enviar sin guía.	InvalidOrderData; CONFIRMED. Confirmar es válido y emite OrderConfirmed. Enviar sin guía se rechaza: queda CONFIRMED y se conserva únicamente ese evento. test_escenario[E11] aprobada.	No se envía debido a que no hay guía, pero se mantiene confirmado el pedido.
E12 Pedido O013	DRAFT; cliente C001, VIP, activo; 1 unidades; total	confirmar dos veces.	InvalidOrderTransition; CONFIRMED. La primera confirmación	Se mantiene la primera confirmación, pero la

	600.00 MXN; sin promoción.		es válida. La segunda se rechaza desde CONFIRMED y no duplica OrderConfirmed. test_escenario[E12] aprobada.	segunda se rechaza al no ser posible.
E13 Pedido O014	DRAFT; cliente C002, no VIP, activo; 1 unidades; total 600.00 MXN; sin promoción.	confirmar, enviar y cancelar.	InvalidOrderTransition ; SHIPPED. Confirmar y enviar con guía son válidos. Cancelar desde SHIPPED se rechaza; conserva los dos eventos anteriores. test_escenario[E13] aprobada.	Se confirma y envía el pedido, pero no se puede cancelar al ya haber sido enviado.
E14 Pedido O015	DRAFT; cliente C005, VIP, activo; 1 unidades; total 450.00 MXN; sin promoción.	confirmar, enviar y devolver.	RETURNED. Confirmar, enviar y devolver son válidos. La regla didáctica permite devolver desde SHIPPED y termina en RETURNED. test_escenario[E14] aprobada.	Se devuelve el pedido sin problema al permitirlo las reglas del negocio.
E15 Pedido O016	DRAFT; cliente C007, VIP, activo; 1 unidades; total 2100.00 MXN; sin promoción.	confirmar, enviar y entregar.	DELIVERED. Confirmar, enviar y entregar son válidos. Termina en DELIVERED con tres eventos de la secuencia. test_escenario[E15] aprobada.	Se entrega el pedido sin ningún problema de por medio.

5. Prueba pytest y captura de resultados

Se realizaron las pruebas con pytest al ejecutar desde la terminal del proyecto "python -m pytest -q". Los resultados fueron los siguientes:



Archivo	Pruebas	Comportamientos comprobados
test_domain.py	52	Money, cantidad, carga, máximo de unidades, inmutabilidad, guía y motivo; matriz de 6 estados × 5 operaciones.
test_specifications .py	11	Reglas independientes, AND/OR/NOT, 500 y 5000 exactos, promoción, prioridad del 10 por ciento y redondeo.
test_scenarios.py	15	E01 a E15 con resultado, estado y secuencia de eventos esperados.
test_api.py	12	HTTP 200/404/409/422, contrato OpenAPI, JSON estricto, estados y aplicaciones independientes.
Total	90	Todas aprobadas en la ejecución adjunta.

La prueba de inventario confirma que la existencia del catálogo no limita el pedido. La frontera de 20 unidades se conserva. También se verifica que un pedido con 19 unidades no reciba dos adicionales y que las listas de artículos y eventos no puedan modificarse desde fuera del agregado.

Los descuentos se prueban sobre el importe bruto. Cuando VIP y monto >5000 cumplen ambas reglas, el resultado es solo 10 por ciento. En presencia de promoción, ambas quedan excluidas.

PoliticaCredito para evaluación normal y PoliticaSurtidoMultiorigen para priorizar ubicación local, almacén central y otras sucursales según BR07. PoliticaDevolucion evalúa categoría y condición cuando FerreMax defina sus parámetros. La última y la de surtido son propuestas de diseño, no clases ya ejecutables.

6. Evidencias de Swagger: operación exitosa y rechazada

Se ejecutó el sistema usando “uvicorn app.api:app --reload” e ingresando el pedido O012. La respuesta fue el código 200: es decir, confirmado. El total se mantuvo en 850, siendo una operación **exitosa**:

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/orders/O012/confirm' \
  -H 'accept: */*' \
  -d ''
```

Request URL

```
http://127.0.0.1:8000/orders/O012/confirm
```

Server response

Code	Details
200	Response body


```
{
  "id": "O012",
  "customer_id": "C012",
  "status": "CONFIRMED",
  "units": 1,
  "currency": "MXN",
  "total": "850.00",
  "discount_percent": "0",
  "discount": "0.00",
  "payable": "850.00",
  "items": [
    {
      "sku": "SKU004",
      "quantity": 1,
      "unit_price": "850.00",
      "subtotal": "850.00"
    }
  ],
  "events": [
    {
      "name": "OrderConfirmed",
      "order_id": "O012",
      "customer_id": "C012",
      "total": "850.00",
      "occurred_at": "2026-10-28T22:32:15.575613+00:00",
      "tracking_number": null,
      "reason": null
    }
  ]
```


Response headers

```
content-length: 420
content-type: application/json
date: Wed,28 Oct 2026 22:32:15 GMT
server: uvicorn
```

Responses

Code	Description
200	Successful Response

Después, se volvió a intentar confirmar el pedido O012, para obtener una ejecución **rechazada** por doble confirmación en un error 409:

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/orders/O012/confirm' \
  -H 'accept: */*' \
  -d ''
```

Request URL

```
http://127.0.0.1:8000/orders/O012/confirm
```

Server response

Code	Details

409	Error: Conflict
	Response body
	`{ "detail": "Requiere DRAFT; estado actual CONFIRMED", "error": "InvalidOrderTransition", "message": "No se puede confirmar el pedido porque ya estáconfirmado. Comprueba su estado antes de volver a confirmarlo." }`
	Response headers
	`content-length: 207`
	`content-type: application/json`
	`date: Wed,28 Oct 2026 22:34:29 GMT`
	`server: uvicorn`

Responses

Code	Description
200	Successful Response

También se intentó confirmar el pedido O007 para obtener una respuesta **rechazada** por un pedido sin artículos en un error 422:

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/orders/O007/confirm' \
  -H 'accept: */*' \
  -d ''
```

Request URL

```
http://127.0.0.1:8000/orders/O007/confirm
```

Server response

Code	Details
422	Error: Unprocessable Content

Response body

```
{
  "detail": "El pedido no contiene artículos",
  "error": "InvalidOrderData",
  "message": "No se puede confirmar un pedido vacío. Agrega al menos un artículo y vuelve a intentarlo."
}
```

Response headers

```
content-length: 176
content-type: application/json
date: Wed,28 Oct 2026 22:36:13 GMT
server: uvicorn
```

Responses

Code	Description
200	Successful Response

Por último, se buscó un pedido inexistente para una operación **rechazada** por inexistencia en un error 404:

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/orders/O023/confirm' \
  -H 'accept: */*' \
  -d ''
```

Request URL

```
http://127.0.0.1:8000/orders/O023/confirm
```

Server response

Code	Details
404	Error: Not Found

Response body

```
{
  "detail": "Pedido inexistente",
  "error": "HTTPException",
  "message": "No se encontró el pedido solicitado. Verifica  e inténtalo nuevamente."
}
```

Response headers

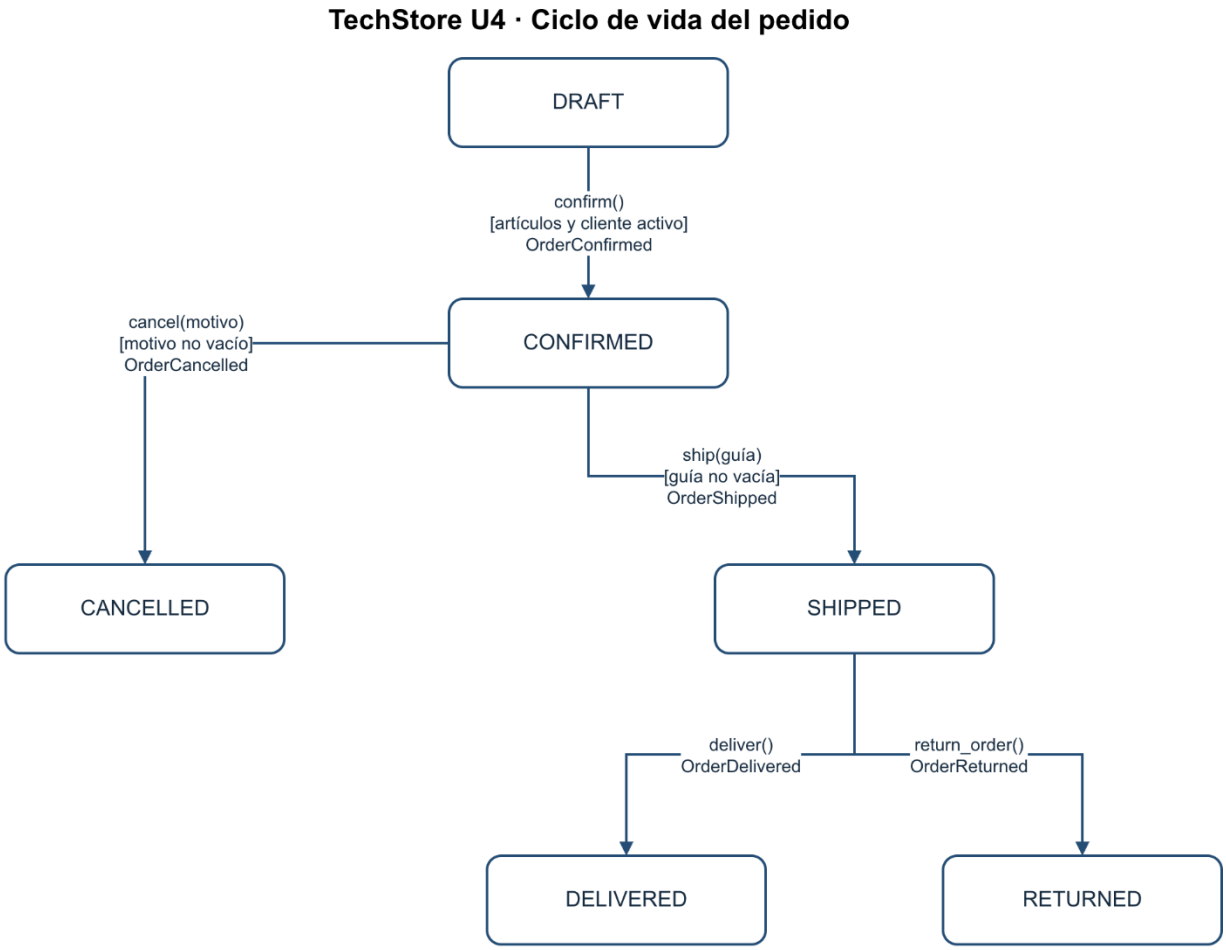
```
content-length: 140
content-type: application/json
date: Wed,28 Oct 2026 22:37:22 GMT
server: uvicorn
```

Responses

Code	Description
200	Successful Response

7. Diagrama de estados y domain events

Este diagrama se encuentra en el repositorio en la carpeta diagramas:



Solo DRAFT permite agregar artículos. Un rechazo conserva estado y eventos.
DELIVERED, CANCELLED y RETURNED no tienen salidas en este laboratorio.

Los eventos del dominio se almacenan dentro del pedido en memoria. Ocurren de la siguiente manera:

Operación	Origen y destino	Condición y evento
confirm()	DRAFT → CONFIRMED	Artículos y cliente activo. Registra OrderConfirmed.
ship(guía)	CONFIRMED → SHIPPED	Guía no vacía. Registra OrderShipped.
cancel(motivo)	CONFIRMED → CANCELLED	Motivo no vacío. Registra OrderCancelled.
deliver()	SHIPPED → DELIVERED	Pedido enviado. Registra OrderDelivered.
return_order()	SHIPPED → RETURNED	Pedido enviado. Registra OrderReturned.

OrderConfirmed comunica que la confirmación ya ocurrió. OrderShipped registra el envío y su guía. OrderCancelled deja constancia del motivo. OrderDelivered representa la entrega y OrderReturned la devolución autorizada por el flujo del laboratorio. Son hechos en pasado, no órdenes para ejecutar una acción.

E11 conserva OrderConfirmed cuando falla el envío sin guía. E12 no duplica ese evento al confirmar otra vez. E13 conserva OrderConfirmed y OrderShipped, pero no añade OrderCancelled. E14 y E15 terminan con tres eventos, uno por cada transición válida.

La matriz de pruebas cubre 30 combinaciones de estado y operación. Solo se aceptan las cinco transiciones de la tabla. Para cada rechazo se compara el estado completo antes y después; para cada éxito se comprueban el destino y el evento recién añadido.

El agregado también protege la composición: agregar artículos solo se permite en DRAFT, los precios deben estar en MXN y la suma de cantidades no puede superar 20. El descuento se evalúa mediante Specifications y no cambia el estado.

8. Reflexión breve: Errores encontrados

Durante la validación se observaron rechazos de negocio y un problema del entorno de ejecución. Los rechazos esperados forman parte de la evidencia: resolverlos significa controlar y comunicar la operación inválida, conservando las reglas del negocio.

Primer error: exceder el límite de unidades

En E08, O009 tenía 19 unidades y se intentaron agregar dos. La operación lanzó OrderItemLimitExceeded. La solución consiste en validar la suma antes de modificar la lista y solicitar una cantidad que no exceda el máximo. La prueba confirmó que se conservaron 19 unidades y el importe previo. El aprendizaje es distinguir la cantidad de renglones de la suma de unidades.

Segundo error: repetir una confirmación

En E12 y en Swagger se llamó confirm() sobre un pedido ya CONFIRMED. Se obtuvo InvalidOrderTransition y HTTP 409. La solución fue proteger el origen permitido y no emitir otro evento. Para repetir una prueba desde el inicio se usa un fixture nuevo o se reinicia el servidor. El aprendizaje es que ejecutar un comando dos veces requiere considerar el ciclo de vida del pedido.

Tercer error: ejecutar pytest con otro intérprete

Al capturar resultados se invocó el intérprete base en lugar del ejecutable del entorno virtual y apareció No module named pytest. Se corrigió usando directamente .venv/bin/python y las dependencias de requirements.txt. La ejecución final produjo 90 pruebas aprobadas. El aprendizaje es comprobar qué intérprete ejecuta el comando, además de verificar que el paquete esté instalado.

9. Preguntas del reporte

¿Por qué se usa Decimal en lugar de float para precios?

Decimal representa los importes decimales sin el error de aproximación binaria típico de float. Se construye desde cadenas, como Decimal("600.00"). Esto permite comparar los umbrales y calcular subtotales de forma consistente. El descuento se redondea explícitamente a centavos con ROUND_HALF_UP.

¿Cuál es la diferencia entre contar renglones y sumar unidades?

Un renglón identifica un artículo con una cantidad. Dos renglones con diez unidades cada uno contienen veinte unidades. El máximo del laboratorio se verifica con sum(item.quantity), no con len(items). E08 muestra que añadir dos a un pedido de diecinueve debe rechazarse aunque haya pocos renglones.

¿Qué ventaja tiene encapsular una regla en Specification?

Permite nombrar, probar y reutilizar condiciones sin modificar el pedido. MinimumAmountSpec, VIPCustomerSpec, MinimumUnitsSpec y ActivePromotionSpec se componen con AND, OR y NOT. Así se separa la elegibilidad de la operación que calcula el importe del descuento, y la misma política puede usarse desde distintas interfaces.

¿Por qué los eventos se registran solo después de una transición válida?

Un evento anuncia un hecho que ya ocurrió. Registrarlo antes de validar podría comunicar un envío sin guía o una confirmación rechazada. La operación valida, cambia el estado y añade el evento. Si falla, conserva el historial anterior. E11, E12 y E13 comprueban expresamente esa ausencia de eventos nuevos.

¿Qué parte del código podrías reutilizar si reemplazas FastAPI por otra interfaz?

Se reutilizarían app/domain.py y app/specifications.py, junto con sus pruebas, porque no dependen de HTTP. Otra interfaz llamaría a confirm(), ship() o quote_discount() con los mismos objetos. Se sustituirían los modelos de entrada y salida y la traducción de excepciones que actualmente pertenecen a app/api.py.

Conclusiones

TechStore convierte los datos de los CSV en objetos que protegen su validez y sus operaciones. Money y OrderItem evitan valores inválidos desde su construcción; Order conserva las reglas de unidades, el estado y los eventos. Las Specifications expresan la política de descuentos sin alterar el pedido.

Los escenarios E01 a E15 comprobaron tanto los éxitos como los rechazos previstos. Las 90 pruebas verificaron las fronteras de monto y cantidad, las transiciones, los eventos y las respuestas de la API. La evidencia incluye la carga de los 16 pedidos, salidas registradas, capturas de Swagger y el diagrama editable.

La API REST ofrece una entrada al mismo dominio que utilizan el cargador, los escenarios y pytest. La persistencia en memoria es suficiente para el objetivo del laboratorio: las reglas y su comprobación son reproducibles, y todos los pedidos vuelven a DRAFT al iniciar una ejecución nueva.

Esta actividad fue una prueba muy útil para recordar en la creación del sistema del proyecto del negocio de nuestro equipo.